

Spread Engine for Treasury Tracking & Liquidity Equalization

Perfect — here's the **end-to-end, DB-first guide** for **all flows** with **physical treasury realization**. I'll use your wallet cache as the “**fiat_currency_by_rate table**” (same thing as `bridge_wallets.fiat_balance_by_rate`): it **pegs** `fiat_value` → (USDC per 1 fiat) `locked_at_time`.

Core objects (no code)

bridge_wallets (crypto wallet rows)

Add:

- **fiat_currency** (VARCHAR(3)) – home display currency for this wallet.
- **fiat_balance_by_rate** (JSONB) – **per-currency buckets** of locked fiat:
 - Single-currency mode (home only): `{"R1": amount_in_home, "R2": amount_in_home, ...}`
 - Multi-currency mode (optional): `{"EUR": {"1.50": 45}, "USD": {"1.01": 50}}`

Each key is a **locked USDC/fiat rate**; each value is **fiat amount locked** at that rate.

transfers (your existing table)

You'll keep using:

- `amount` (string), `currency` (string), `source` (JSON), `destination` (JSON), `state`, timestamps...
- Put all FX math + treasury settlement details into **receipt JSON** (e.g., `consumed_buckets`, `locked_rate_used`, `live_rate_used`, `amount_usdc`, `delta_usdc`, `treasury_pnl_usdc`, notes).

Treasury wallet

A corporate **USDC** wallet. We **physically move Δ USDC** between **user**↔**treasury** on every send/withdraw where there is a spread.

How to compute the spread on a home-currency burn

When you “consume” Y units of a wallet’s home currency H :

- $\text{USDC_locked} = \sum (\text{fiat_used_i} \times \text{locked_rate_i})$ from consumed buckets
- $\text{USDC_live} = Y \times \text{live_rate_H_at_execution}$
- $\Delta_{\text{USDC}} = \text{USDC_live} - \text{USDC_locked}$
 - $\Delta > 0 \rightarrow$ (*compensation*) treasury **sends Δ** to the user wallet
 - $\Delta < 0 \rightarrow$ (*profit*) user wallet **sends $|\Delta|$** to treasury

You then fund the downstream leg with the **right amount of USDC** (see each flow).

1) Regular deposit (bank H $\rightarrow$ wallet H)

On-chain USDC: Mint $X \times R_{H_live}$ to the **user wallet**.

Wallet cache: Add $+X$ to bucket “ **R_{H_live}** ”.

Treasury move: None (locked at live; no spread).

transfers (1 row):

- **source:** {**bank_account**, **currency:**H}
- **destination:** {**wallet_id:**user_wallet, **currency:**USDC}
- **amount:** **X** , **currency:** H

- `receipt: {leg:"fiat_to_usdc", currency_pair:"H_USDC", live_rate_used:R_H_live, locked_rate_used:R_H_live, amount_usdc:X*R_H_live, treasury_pnl_usdc:0}`
-

2) Regular withdrawal (wallet H → bank H)

Goal: Pay Y H to bank at **live**. Bank/off-ramp needs **USDC_live**.

1. **Burn H buckets** in wallet cache: $-Y$ (lowest locked first if you optimize P&L).
2. Compute **USDC_locked**, **USDC_live**, Δ .
3. **Treasury settlement (physical):**
 - $\Delta > 0$ (comp): **treasury** → **user** Δ **USDC**
 - $\Delta < 0$ (profit): **user** → **treasury** $|\Delta|$ **USDC**
(This guarantees the user can now send exactly **USDC_live** to the off-ramp.)
4. **USDC to bank: user** → **off-ramp** **USDC_live** (off-ramp pays Y H).

Wallet cache: After step 1: buckets reflect $-Y$; drop empty keys.

transfers (2 rows):

- **Row A (treasury_settlement):** moves Δ **USDC** between **user** ↔ **treasury**
 - `receipt: {leg:"treasury_settlement", currency_pair:"H_USDC", consumed_buckets:[{rate,fiat}], locked_rate_used:(avg or per-bucket), live_rate_used:R_H_live, delta_usdc: $\Delta$ , treasury_pnl_usdc: $\Delta$ }`
- **Row B (usdc_to_fiat):** **user** → **bank**, **USDC_live**
 - `receipt: {leg:"usdc_to_fiat", currency_pair:"USDC_H", live_rate_used:R_H_live, amount_usdc:USDC_live}`

3) Foreign deposit (bank F → wallet H)

On-chain USDC: Mint $X_F \times R_{F_live}$ to user wallet.

Credit home H in cache: $H_amount = (X_F \times R_{F_live}) \div R_{H_live_credit}$; add +H_amount under “R_H_live_credit”.

Treasury move: None (deposit locks at current credit rate; spread realized later when funds are spent).

transfers (1 row):

- `source: {bank_account, currency:F}`
- `destination: {wallet_id:user_wallet, currency:USDC}`
- `amount: X_F, currency: F`
- `receipt: {leg:"fiat_to_usdc", currency_pair:"F_USDC", live_rate_used:R_F_live, amount_usdc:X_F*R_F_live, secondary_credit:{currency:H, credited_amount:H_amount, lock_rate:R_H_live_credit}, treasury_pnl_usdc:0}`

4) Foreign withdrawal (wallet H → bank F)

User asks for Y in home H (you promise exact Y H equivalent in F at live F).

1. **Burn H buckets:** $-Y$; compute `USDC_locked`, `USDC_live_H = Y × R_H_live`; $\Delta = \text{USDC_live_H} - \text{USDC_locked}$.
2. **Treasury settlement (physical):**
 - $\Delta > 0$: **treasury** → **user** Δ USDC
 - $\Delta < 0$: **user** → **treasury** $|\Delta|$ USDC

3. **Payout F using live F:** send **USDC_live_H** from user → off-ramp; off-ramp pays $F_amount = USDC_live_H \div R_F_live$ to bank.

Wallet cache: reflects $-Y$ in H; drop empty keys.

transfers (2 rows):

- **Row A (treasury_settlement):** as in regular withdrawal, with `currency_pair:"H_USDC"`, `treasury_pnl_usdc: $\Delta$` .
 - **Row B (usdc_to_fiat foreign):** user → bank_F, `amount_usdc = USDC_live_H`
 - `receipt: {leg:"usdc_to_fiat", currency_pair:"USDC_F", live_rate_used:R_F_live, amount_usdc:USDC_live_H, reference_home:{requested:Y, home_live:R_H_live}}`
-

5) Same-currency send (wallet H → wallet H)

Sender wants to send Y H to recipient in the same currency.

1. **Burn sender H buckets:** $-Y$; compute **USDC_locked**, $USDC_live = Y \times R_H_live$, Δ .
2. **Fund recipient:** user → recipient **USDC_locked** (this is the USDC corresponding to the locked buckets).
3. **Treasury settlement (physical):**
 - $\Delta > 0$ (comp): treasury → user Δ USDC
 - $\Delta < 0$ (profit): user → treasury $|\Delta|$ USDC
(Sender's *total* USDC debited = $USDC_locked + \max(\Delta, 0) = USDC_live$.)
4. **Credit recipient cache:** $+Y$ under recipient's new locked rate **R_H_live_credit**.

Wallet caches:

- **Sender H:** -Y (drop empty keys)
- **Recipient H:** +Y at bucket "R_H_live_credit"

transfers (3 rows):

- **Row A (wallet_to_wallet):** sender → recipient, amount_usdc = USDC_locked
 - receipt: {leg:"wallet_to_wallet", currency_pair:"H_USDC", consumed_buckets:[{rate,fiat}], live_rate_used:R_H_live, amount_usdc:USDC_locked, treasury_pnl_usdc:0}
- **Row B (treasury_settlement):** user↔treasury, Δ USDC
 - receipt: {leg:"treasury_settlement", currency_pair:"H_USDC", live_rate_used:R_H_live, locked_rate_used:(avg or per-bucket), delta_usdc:Δ, treasury_pnl_usdc:Δ}
- **Row C (usdc_to_fiat credit):** recipient self-credit of Y at R_H_live_credit
 - receipt: {leg:"usdc_to_fiat", currency_pair:"USDC_H", live_rate_used:R_H_live_credit, amount_usdc:USDC_locked, locked_rate_used:R_H_live_credit, treasury_pnl_usdc:0}

6) Cross-currency send (wallet C_s → wallet C_r)

Sender wants to send Y in C_s; recipient holds C_r.

1. **Burn sender C_s buckets:** -Y; compute USDC_locked, USDC_live = Y × R_C_s_live, Δ.
2. **Fund recipient:** sender → recipient USDC_locked.
3. **Treasury settlement (physical):**

- $\Delta > 0$: treasury $\rightarrow$ sender Δ
 - $\Delta < 0$: sender $\rightarrow$ treasury $|\Delta|$
(Sender's total USDC debit equals USDC_live.)
4. Recipient credit in C_r : $C_r_amount = USDC_locked \div R_C_live_credit$; $+C_r_amount$ at bucket " $R_C_live_credit$ ".

Wallet caches:

- Sender C_s : $-Y$ (drop empty keys)
- Recipient C_r : $+C_r_amount$ at $R_C_live_credit$

transfers (3 rows):

- Row A (wallet_to_wallet): sender $\rightarrow$ recipient, amount_usdc = USDC_locked
 - receipt: {leg:"wallet_to_wallet", currency_pair:"C_USDC", consumed_buckets:[{rate,fiat}], live_rate_used:R_C_live, amount_usdc:USDC_locked, treasury_pnl_usdc:0}
- Row B (treasury_settlement): user $\leftrightarrow$ treasury, Δ USDC
 - receipt: {leg:"treasury_settlement", currency_pair:"C_USDC", live_rate_used:R_C_live, locked_rate_used:(avg/per-bucket), delta_usdc: Δ , treasury_pnl_usdc: Δ }
- Row C (usdc_to_fiat credit): recipient self-credit in C_r , amount_usdc = USDC_locked
 - receipt: {leg:"usdc_to_fiat", currency_pair:"USDC_Cr", live_rate_used:R_C_live_credit, locked_rate_used:R_C_live_credit, amount_usdc:USDC_locked, treasury_pnl_usdc:0}

What the fiat_currency_by_rate table is doing for you

- It **pegs** each **fiat amount** to the **exact USDC/fiat rate at the time** of deposit/credit.
- When spending, you **burn** those pegged amounts and convert to **USDC_locked** deterministically.
- The **spread vs live** is **always** realized **physically** via the **treasury_settlement** row (USDC moves user↔treasury).
- The downstream leg (recipient credit or bank payout) is **funded by the correct USDC** (USDC_locked for wallet-to-wallet, **USDC_live** for bank payouts).

Ledger truth

- Each economic leg = **one transfers row** with full detail inside **receipt**.
- **Sends** (same/cross-currency) = **3 rows** (fund recipient, treasury settlement, recipient credit).
- **Deposits** = 1 row.
- **Withdrawals** = **2 rows** (treasury settlement, off-ramp).
- **Treasury P&L** is exactly **treasury_pnl_usdc = delta_usdc** on the **treasury_settlement** row:
 - **positive** = profit (user → treasury)
 - **negative** = compensation (treasury → user)

This is the consistent pattern that keeps your **wallet cache** precise, your **recipient funding** correct, and your **treasury economics** realized on-chain every time.